Industries & Regulations RAG Architecture MongoDB Guide

A Comprehensive Technical Reference

Created: June 22, 2026

1. INDUSTRIES & THEIR REGULATIONS

1.1 Financial Services & Banking

The financial industry operates under strict regulatory frameworks to ensure stability, transparency, and consumer protection.

- Basel III: International banking regulations governing capital adequacy and risk management
- GDPR (EU): Protects customer data privacy with strict consent and data handling requirements
- SOX (Sarbanes-Oxley): Mandates financial reporting transparency and internal controls
- PCI-DSS: Payment Card Industry Data Security Standard for secure payment processing
- Know Your Customer (KYC): Anti-money laundering protocols requiring customer verification

1.2 Healthcare & Pharmaceuticals

Healthcare is one of the most regulated industries due to patient safety and data sensitivity concerns.

- HIPAA: Health Insurance Portability and Accountability Act protecting patient medical records
- FDA Regulations: Food and Drug Administration oversight of medications and medical devices
- Clinical Trial Standards: GCP (Good Clinical Practice) for ethical clinical research
- EHR Compliance: Electronic Health Records must maintain integrity and auditability
- Pharmacovigilance: Post-market surveillance of drug safety and adverse reactions

1.3 Aviation & Transportation

Safety-critical industries with severe regulatory oversight to prevent accidents and ensure operational reliability.

- FAA Regulations: Federal Aviation Administration standards for aircraft safety and maintenance
- ISO 9001: Quality management systems for transportation operators
- TSA Security: Transportation Security Administration protocols for airport and aviation security
- DOT Requirements: Department of Transportation safety standards for all transport modes

1.4 Energy & Utilities

Energy sector regulations ensure grid stability, environmental protection, and consumer safety.

- NERC Standards: North American Electric Reliability Corporation for power grid reliability
- EPA Environmental: Environmental Protection Agency emissions and waste management standards
- Nuclear Regulations: NRC oversight of nuclear power plants and radioactive material handling
- Safety & Compliance: Workplace safety standards (OSHA) and hazard management

1.5 Food & Beverage Industry

Food safety regulations protect consumers from contamination and ensure nutritional transparency.

- FDA Food Safety: Prevents foodborne illness through manufacturing and handling standards
- HACCP: Hazard Analysis and Critical Control Points for food safety management
- Labeling Standards: Nutritional information and allergen disclosure requirements
- Hygiene Codes: Local and international food preparation and storage standards

1.6 Data Protection & Privacy (Cross-Industry)

All industries handling personal data must comply with increasingly strict privacy regulations.

- GDPR: EU General Data Protection Regulation with €20M fines for violations
- CCPA: California Consumer Privacy Act giving users data rights and deletion options
- ISO 27001: Information Security Management System certification
- Data Breach Notification: Legal requirement to notify users of data compromises within days

2. RETRIEVAL-AUGMENTED GENERATION (RAG) ARCHITECTURE

RAG is a hybrid AI architecture that combines large language models (LLMs) with external knowledge retrieval systems. Instead of relying solely on model parameters, RAG retrieves relevant documents from a knowledge base to augment the model's generation capability.

2.1 Why RAG?

- Reduces Hallucination: Grounding responses in retrieved documents improves accuracy
- Current Information: Retrieves up-to-date data without retraining the LLM
- Cost Efficient: Smaller models can perform like larger ones when augmented with retrieval
- Domain Specific: Easily adapt to custom knowledge bases (medical records, legal docs, etc.)

2.2 Core Components of RAG

1. Data Ingestion & Preprocessing

- Document Collection: Gather documents (PDFs, webpages, databases, logs)
- Chunking: Split large documents into semantic chunks (256-1024 tokens typical)
- Cleaning: Remove noise, normalize formatting, handle special characters
- Metadata Extraction: Preserve source, date, author, and document type information

2. Embedding & Vector Representation

- Embedding Model: Convert text chunks into dense vectors (e.g., OpenAI, Sentence-BERT)
- Vector Dimension: Typical sizes are 768, 1024, or 1536 dimensions
- Semantic Meaning: Embeddings capture semantic similarity between chunks
- Indexing: Store vectors in vector database for fast similarity search

3. Vector Database (Knowledge Store)

- Pinecone: Cloud-based vector database with built-in indexing
- Weaviate: Open-source vector search engine with flexible filtering
- Milvus: Scalable vector database designed for massive datasets
- Faiss: Facebook AI Similarity Search for efficient retrieval at scale
- ChromaDB: Lightweight, in-memory vector store for rapid prototyping

4. Retrieval Mechanism

- Query Embedding: Convert user query to same vector space as document chunks
- Similarity Search: Find top-k most similar chunks (k typically 3-10)
- Ranking: Re-rank retrieved documents by relevance using cross-encoders
- Filtering: Apply metadata filters (date range, source, document type)

5. Context Augmentation

- Retrieved Context: Combine top-k documents into context window

• Prompt Construction: Insert context into system prompt with clear boundaries

• Chain-of-Thought: Structure prompts to encourage reasoning over retrieved facts

6. Generation by LLM

• LLM Processing: Model receives augmented prompt with context + user query

• Response Generation: Model generates response grounded in retrieved documents

• Citation: Include source attribution for transparency and traceability

2.3 RAG Pipeline Workflow

Step	Action	Input	Output	
1	User Query	Natural language question	Query text	
2	Embed Query	Text embedding model	Query vector (768-1536 dims)	
3	Vector Search	Top-k similarity search	Top 5-10 relevant chunks	
4	Rank Results	Cross-encoder or LLM ranking	Sorted, filtered documents	
5	Format Context	Combine with prompt template	Augmented prompt	
6	Generate	LLM inference on augmented input	Response with citations	
7	Return	Present to user	Response + source docs	

2.4 Advanced RAG Techniques

• Multi-Query RAG: Generate multiple queries from one user input for comprehensive retrieval

• Hierarchical RAG: Use coarse-to-fine retrieval (summary → detailed chunks)

• Hybrid Search: Combine semantic (vector) + keyword (BM25) search for better recall

• Re-ranking: Use cross-encoder models to re-rank retrieved documents

• Query Expansion: Expand queries with synonyms and related terms

• Feedback Loop: Use user feedback to improve retrieval quality over time

2.5 RAG Performance Metrics

• Retrieval Precision: % of retrieved documents that are relevant to query

• Retrieval Recall: % of all relevant documents that were retrieved

• MRR (Mean Reciprocal Rank): Average rank of first relevant document

• NDCG (Normalized Discounted Cumulative Gain): Ranking quality metric

• Generation Quality: BLEU, ROUGE scores for generation evaluation

• Hallucination Rate: % of generated content not supported by retrieved docs

3. MONGODB: NOSQL DATABASE GUIDE

MongoDB is a leading NoSQL document database designed for flexibility, scalability, and developer productivity. Unlike relational databases, MongoDB stores data as flexible JSON-like documents in collections.

3.1 Key Differences: MongoDB vs SQL

Aspect	SQL (Relational)	MongoDB (NoSQL)
Data Structure	Fixed tables with rows/columns	Flexible JSON documents in collections
Schema	Rigid schema enforced upfront	Flexible schema, schema-on-write or schema-on-read
Transactions	ACID within single database	ACID transactions in v4.0+ (multi-document)
Scalability	Vertical (more powerful server)	Horizontal (sharding across multiple servers)
Joins	Multi-table joins via SQL	Aggregation pipelines, embedding, or denormalization
Query Language	SQL (declarative)	Aggregation framework (imperative)
Best For	Structured, relational data	Unstructured, semi-structured, evolving schemas

3.2 Core Concepts

Database

Top-level container holding multiple collections. One MongoDB server can host many databases.

Collection

Equivalent to SQL tables. Collection groups related documents together without enforcing a fixed schema.

Document

Equivalent to SQL rows. Documents are BSON (Binary JSON) objects with key-value pairs. Each document has a unique _id.

BSON (Binary JSON)

MongoDB's internal data format. BSON extends JSON with additional types: Date, Binary, ObjectId, etc.

3.3 Data Types in MongoDB

- String: Text values
- Number: Integer, Long, Double (32-bit, 64-bit, floating point)
- Boolean: true/false
- Date: ISO 8601 datetime format
- Array: Ordered lists of values [value1, value2, ...]
- Object: Nested documents { key: value }
- Null: Null values
- ObjectId: 12-byte unique identifier (default _id)
- Binary: Binary data (images, files)

3.4 Indexing in MongoDB

Indexes speed up query execution by creating a sorted data structure.

- Single Field Index: Index on one field { name: 1 }
- Compound Index: Index on multiple fields { name: 1, age: -1 }
- Text Index: Full-text search capability on string fields
- Geospatial Index: Queries on geographic coordinates (2dsphere, 2d)
- TTL Index: Auto-delete documents after expiration time
- Unique Index: Enforce uniqueness on a field (e.g., email)

3.5 Aggregation Pipeline

Powerful framework for data transformation and analysis using stages (like Unix pipes).

- $match: Filter documents (like WHERE in SQL)
- $group: Group documents and compute aggregates (SUM, AVG, COUNT)
- $sort: Sort documents by field
- $project: Reshape documents (select, compute fields)
- $lookup: Join collections (like SQL JOIN)
- $unwind: Flatten arrays into multiple documents
- $facet: Multi-faceted analysis in one pass

3.6 CRUD Operations

Create

Insert documents: db.users.insertOne({name: 'John', age: 30})

Read

Query documents: db.users.find({age: {$gt: 25}}) — Find users older than 25

Update

Modify documents: db.users.updateOne({_id: id}, {$set: {age: 31}})

Delete

Remove documents: db.users.deleteOne({_id: id})

3.7 Schema Design Patterns

Embedding Pattern

Nest related data inside a document (denormalization). Fast for reads, slower for writes.

Referencing Pattern

Store document IDs instead of full documents. Requires $lookup joins, good for frequently changing data.

Hybrid Pattern

Combine embedding (essential data) + referencing (optional/large data) for optimal performance.

3.8 Replication & Sharding

Replication (High Availability)

Replica Set: Multiple copies of data across servers. Primary (write) + Secondaries (read). Auto-failover on primary failure.

Sharding (Horizontal Scalability)

Distribute data across multiple servers by shard key. Each shard holds a subset of data. Transparent to application.

3.9 Use Cases

- Content Management: Blog posts, articles with flexible metadata
- User Profiles: Complex user data with varying attributes
- Time-Series Data: IoT sensors, metrics, logs at scale
- Mobile Apps: Offline-first with sync capability
- Real-time Analytics: Event streaming, document updates
- E-commerce: Product catalogs with varied attributes

3.10 MongoDB vs Alternatives

- vs Cassandra: MongoDB for consistency, Cassandra for ultra-high availability
- vs PostgreSQL: PostgreSQL for relational integrity, MongoDB for flexibility
- vs Elasticsearch: MongoDB for general storage, Elasticsearch for full-text search
- vs DynamoDB: MongoDB is self-hosted, DynamoDB is AWS managed service

---

This document provides foundational knowledge on Industries & Regulations, RAG Architecture, and MongoDB. For production deployments, consult official documentation and security experts.